UNIVERSITY OF GHANA

DEPARTMENT OF COMPUTER SCIENCE

DCIT 204/308: Data Structures and Algorithms I & II

Joint DSA Semester Project — Technical Report

Ghana Smart Service Operations Optimizer: A Courier & Food Delivery Dispatch System for Accra

Name	Index Number	Oral-Defense Topics
Immanuel Oheneba Debe	22243130	Disjoint Set (Union-Find) & Kruskal's MST
Jonas Kudzo Amuzu	22198544	Red-Black Tree & Dynamic Programming (Knapsack)
Cedric Dzodzodzi	22046156	Hash Table & Greedy Dispatcher
Bilson Priscilla Essirifua	22015128	BST & Dijkstra's Shortest Path
Emerald Aryee	22032619	B-Tree & Prim's MST
Meshach Ashitei Amarh	22231662	Priority Queue (Heap) & Merge Sort
Bamfo Kwabena Asante-Darko	22262799	Dynamic Array & Quicksort
Frederick Addison	22310210	Linked List + Iterator & Binary Search
Lawrencia Ametu	22413790	Stack & Linear Search
Gladys Anobea Antwi	22376230	Circular Queue & Selection Sort
Bernard Tetteh	22392052	Deque & Insertion Sort
Samuel Osei-Bonsu	22380629	Custom Set / Map & BFS
Roland Otabil Kwesi	22268717	Graph (Adjacency List & Matrix) & DFS
Oduah Chisom	22254852	Brute-Force/Exhaustive Search & Asymptotic Notation (Big-O/Θ/Ω)
Eduful Eugene Nana Yaw Addo	22383968	Efficiency Analysis & Correctness/Tractability Proof

		Sketches

Local Context: Courier / Food Delivery Service — Accra, Ghana

Table of Contents

1. Problem Statement, Assumptions, and System Boundaries

1.1 Local Context

The system models a courier and food delivery service operating across Accra, Ghana. The organisation receives delivery requests from vendors (restaurants, pharmacies, shops) destined for customer zones across the city, and must dispatch a limited pool of riders, bicycles, tricycles, and vans to fulfil these requests under real-world constraints: road conditions, distances, urgency levels, and limited vehicle capacity.

1.2 Core Operational Questions

- Which pending delivery request should be assigned next — under FIFO, urgency-based, or priority-weighted dispatch rules?
- What is the fastest route between two locations in the city, accounting for distance, travel time, and road condition?
- Which locations are reachable from a given rider hub or dispatch point?
- Given a resource's limited carrying capacity, which subset of pending requests should it be assigned to maximise value delivered?
- How do alternative data structures and algorithms (e.g. BST vs Red-Black Tree, linear vs binary search) perform as the request volume grows?
- How can the system persist operational data (locations, roads, requests, resources) so it can be reloaded and analysed later?

1.3 Assumptions

- Road weights (distance, travel time, condition) are static for the duration of a dispatch cycle — no live traffic feed.
- A request has exactly one source (vendor) and one destination (customer zone).
- Resources (riders/vehicles) operate from a fixed home location and have a fixed capacity.
- All seed data is synthetic but structurally realistic (Accra-area names, plausible distances) — see Section 2.4.

1.4 System Boundaries

The system is a console-driven Java application backed by a MySQL database. It is not a UI/UX project: there is no web or mobile front end. The console menu exists so an examiner can run every module directly without editing source code. The system does not perform live GPS tracking, payment processing, or real-time traffic ingestion.

2. Dataset Description, Data Dictionary, and Database Schema

2.1 Dataset Summary

Entity	Minimum Required	Actual Seeded
Locations	50	67
Roads / Edges	100	110
Service Requests	300	310
Resources	30	32

Algorithm Runs	30	216 (after full Performance Lab run)

2.2 Data Dictionary

locations

Field	Type	Description
location_id	VARCHAR(10)	Primary key, e.g. L001
name	VARCHAR(120)	Place name
area	VARCHAR(80)	Neighbourhood, e.g. Osu, Madina, East Legon
location_type	ENUM	Vendor, CustomerZone, RiderHub, or Landmark
x_coord / y_coord	DECIMAL(9,6)	Latitude / longitude

roads

Field	Type	Description
road_id	VARCHAR(10)	Primary key, e.g. R001
from_location_id / to_location_id	VARCHAR(10)	Foreign keys into locations
distance_km	DECIMAL(6,2)	Road segment distance
travel_time_min	DECIMAL(6,2)	Expected travel time
condition_weight	DECIMAL(4,2)	1.0 = good road; higher = worse/slower

service_requests

Field	Type	Description
request_id	VARCHAR(10)	Primary key, e.g. Q001
source_location_id / destination_location_id	VARCHAR(10)	Foreign keys into locations
category	ENUM	Food, Parcel, Document, Grocery, Pharmacy, Medical
urgency	TINYINT	1 (low) to 5 (critical)
time_submitted / deadline	DATETIME	Request timing window
status	ENUM	NEW, ASSIGNED, IN_TRANSIT, DELIVERED, CANCELLED

resources

Field	Type	Description
resource_id	VARCHAR(10)	Primary key, e.g. V001, B001, T001
resource_type	ENUM	Rider, Bicycle, Tricycle, Van
home_location_id	VARCHAR(10)	Foreign key into locations
capacity	INT	Number of parcels it can carry
availability_status	ENUM	AVAILABLE, BUSY, OFFLINE

algorithm_runs / audit_events

algorithm_runs stores empirical timing measurements (algorithm_name, input_size, time_ns, date_run) from the Performance Lab. audit_events is a stack-based undo/audit log of status changes and assignment events (event_type, old_value, new_value).

2.3 Index-Number-Derived Parameters

Per the brief's requirement to derive at least three algorithm parameters from team member index numbers, the following are used throughout the system (see ProjectParameters.java):

Parameter	Value	Derivation
HASH_TABLE_SIZE	31	From Immanuel's index (22243130): last 2 digits = 30, rounded up to the nearest prime (31) to reduce clustering.
RANDOM_SEED	830	Sum of the last 3 digits of three team members' index numbers (Immanuel, Jonas, Cedric): 130 + 544 + 156 = 830. Used to make the seed dataset and benchmark runs reproducible.
DEFAULT_VEHICLE_CAPACITY	7	From Jonas's index (22198544): last 2 digits = 44, 44 % 10 = 4, plus a base capacity of 3 = 7.

2.4 Evidence Note on Dataset Construction

No real vendor, customer, or rider identities were used anywhere in this project. All 67 locations, 110 roads, 310 service requests, and 32 resources were synthetically generated, with two sources of realism: place names and areas (Osu, Madina, East Legon, Circle, Dansoman, etc.) were drawn from publicly known Accra neighbourhoods and landmarks — general local knowledge, not any individual's private data; road distances, travel times, and condition weights were assigned plausible synthetic values reflecting typical Accra traffic patterns (shorter inner-city links get lower travel times, outer links get higher distance and condition-weight values to simulate less maintained roads); service request categories (Food, Parcel, Document, Grocery, Pharmacy, Medical) and urgency levels (1–5) were chosen to reflect a realistic courier/delivery mix; resource types (Rider, Bicycle, Tricycle, Van) and capacities were set to plausible real-world values for a small Accra delivery operation. Random generation used RANDOM_SEED = 830 (Section 2.3), so the full dataset is reproducible from ProjectParameters.java without needing to store or share a raw generation script separately.

3. System Architecture and Module Design

3.1 High-Level Architecture

The system follows a layered console architecture: a MySQL database persists all operational data; a DataLoader reads records into custom in-memory data structures; algorithm engines (search/sort, graph, scheduling, optimisation) operate on those structures; and a console menu (Main.java / ConsoleUI.java) exposes every module to the examiner without requiring source edits.

3.2 Console Menu Structure

Menu	Options

1. Data Loading & Database	Load locations, load service requests, show DB status
2. Custom Data Structures	Linked List+Iterator, Stack, Queue/Circular Queue, Deque, Priority Queue/Heap, BST, Hash Table, Dynamic Array, Red-Black Tree, B-Tree, Custom Set/Map, Disjoint Set, Adjacency Matrix
3. Searching & Sorting	Linear/binary search and all four sorts, demonstrated together
4. Graph Algorithms	BFS & DFS traversal, Dijkstra shortest path, Kruskal vs Prim MST, Adjacency Matrix demo
5. Dispatch Scheduling	FIFO and urgency/priority-based dispatch demonstrations
6. Optimisation	Greedy nearest-rider dispatch vs Dynamic Programming knapsack, including a greedy-failure counterexample
7. Performance Lab	Runs all 6 empirical benchmark suites, exports CSV, saves to algorithm_runs
8. Integrated Dispatch	AUTO and INTERACTIVE end-to-end dispatch against live pending requests

3.3 Package Layout

- gh.dso.datastructures — custom list, stack, deque, heap, tree (BST/RedBlackTree/BTree), hash, set packages
- gh.dso.algorithms — search and sort implementations
- gh.dso.graph — Graph, GraphTraversal, Dijkstra, Kruskal, Prim, DisjointSet
- gh.dso.scheduling — FIFO/priority dispatch queues
- gh.dso.optimization — GreedyDispatcher, KnapsackOptimizer
- gh.dso.db — DatabaseConnection, DataLoader, SeedImporter
- gh.dso.performance — PerformanceLab (empirical benchmarking harness)
- gh.dso.model — Location, Road, ServiceRequest, Resource, AlgorithmRun records

4. Data-Structure Implementation

All 12 required structures (Section 6 of the brief) are implemented from scratch — no java.util.HashMap, TreeMap, PriorityQueue, Stack, or ArrayDeque used for assessed logic. Each has a dedicated JUnit test class covering normal, boundary, and invalid-input cases, part of the 131-test suite:

Structure	Key Operations	Test Class
Dynamic Array	insert, get, set, remove, auto-resize (doubling)	MyDynamicArrayTest
Linked List + Iterator	addFirst, addLast, insertAfter, remove, iterator	MyLinkedListTest
Stack	push, pop, peek, isEmpty (array-backed)	MyStackTest
Queue / Circular Queue	enqueue, dequeue, wrap-around handling	CircularQueueTest
Deque	addFront, addRear, removeFront, removeRear	MyDequeTest
Priority Queue (Heap)	insert, extract, heapify (binary heap)	MyPriorityQueueTest
BST	insert, search, inorder traversal, searchPathLength	BSTTest
Red-Black Tree	insert with rotation/recolouring, search	RedBlackTreeTest

B-Tree	node split, search	BTreeTest
Hash Table	put, get, remove, collision chaining, loadFactor()	MyHashTableTest
Custom Set / Map	membership, lookup (built on hash table)	MySetTest / MyMapTest
Disjoint Set	makeSet, find (path compression), union	DisjointSetTest
Graph	adjacency list + matrix, BFS/DFS/Dijkstra/MST	GraphTraversalTest / GraphMatrixTest / DijkstraTest / MstTest

Section 6 below gives trace-level evidence (not just pass/fail) for the structures most central to the dispatch problem: BST/Red-Black Tree, Disjoint Set, and the Hash Table.

5. Algorithm Implementation

5.1 Searching & Sorting

LinearSearch scans sequentially, O(n), no precondition. BinarySearch (SearchAlgorithms.java) computes mid = lo + (hi − lo) / 2 each iteration and requires ascending-sorted input — its precondition and failure mode are traced in Section 6.11. SelectionSort and InsertionSort are both $O(n^2)$ in-place comparison sorts; InsertionSort is adaptive (near-O(n) on nearly-sorted data) while SelectionSort is not. MergeSort (top-down, recursive split at mid) guarantees O(n log n) with O(n) auxiliary space and is stable. QuickSort uses Lomuto partitioning with the last element as pivot, O(n log n) average but $O(n^2)$ on already-sorted input since no pivot randomisation is applied.

5.2 Graph Algorithms

BFS and DFS traverse the adjacency-list representation in O(V+E). Dijkstra (heap-based) computes single-source shortest paths in O((V+E) log V), assuming non-negative weights — true here since Road.effectiveWeight() = travelTimeMin × conditionWeight is always positive. Kruskal sorts all edges by weight and uses the Disjoint Set to reject edges that would form a cycle, O(E log E). Prim grows a single tree from a start node using a priority queue, O(E log V).

5.3 Optimisation: Greedy vs Dynamic Programming

GreedyDispatcher assigns the nearest available rider to each request in arrival order — fast, locally optimal, but not globally optimal under capacity constraints (Section 6.11 proves this). KnapsackOptimizer solves the 0/1 knapsack exactly via bottom-up DP tabulation: dp[i][c] = max(dp[i−1][c], dp[i−1][c−weight(i)] + value(i)), with items = pending requests, weight = category-derived weight, value = urgency, and capacity = DEFAULT_VEHICLE_CAPACITY.

Full trace tables, a loop-invariant proof (binary search), an induction proof (merge sort), and a DP optimal-substructure proof are given in Section 6 below, all computed directly from this project's real seed data and test cases.

6. Correctness and Testing Evidence

6.1 Unit Test Summary

The full JUnit suite passes 131/131 tests across all data structures and algorithms, with normal, boundary, and invalid-input cases per structure, per the brief's Section 10 requirement.

6.2 A Real Bug Found and Fixed

During integration testing, MstTest.kruskal_disconnectedGraph_invalidCase_fewerThanNMinus1Edges failed with a NullPointerException. Root cause: Kruskal.buildMst did not validate that a road's endpoints were part of the given location set before calling DisjointSet.union — when a road referenced a location outside the disconnected test's location set, DisjointSet.find threw an NPE on the unregistered id rather than failing cleanly.

Fix: Kruskal.buildMst now filters roads to only those whose both endpoints are in the known location set before inserting them into the edge priority queue, and DisjointSet.find now throws a clear IllegalArgumentException on an unregistered id instead of a bare NullPointerException. This is itself an instance of the brief's required 'invalid precondition' edge case (Section 10).

All traces below use real values from the project's own seed data (database/seed/roads.csv, service_requests.csv) or the project's own JUnit test cases — not generic textbook arrays.

6.3 Trace Table 1: Binary Search

Array (sorted ascending distance_km from roads R001–R009): [0.89, 1.64, 1.87, 2.72, 3.18, 4.05, 6.46, 7.59, 9.18]. Target = 9.18 (worst case for this array).

lo	hi	mid	A[mid]	Compare to 9.18	Action
0	8	4	3.18	3.18 < 9.18	lo = mid + 1 = 5
5	8	6	6.46	6.46 < 9.18	lo = mid + 1 = 7
7	8	7	7.59	7.59 < 9.18	lo = mid + 1 = 8
8	8	8	9.18	9.18 = 9.18	FOUND at index 8 (4 comparisons)

6.4 Trace Table 2: Insertion Sort

Array (real travel_time_min from roads R001–R006): [2.10, 8.09, 9.53, 22.98, 10.48, 3.98]

Pass (i)	Key	Comparisons	Shifts	Array state after pass
1	8.09	1	0	[2.10, 8.09, 9.53, 22.98, 10.48, 3.98]
2	9.53	1	0	[2.10, 8.09, 9.53, 22.98, 10.48, 3.98]
3	22.98	1	0	[2.10, 8.09, 9.53, 22.98, 10.48, 3.98]
4	10.48	2	1	[2.10, 8.09, 9.53, 10.48, 22.98, 3.98]
5	3.98	5	4	[2.10, 3.98, 8.09, 9.53, 10.48, 22.98]

Totals: 10 comparisons, 5 shifts — matching the SortStats(comparisons, swaps) instrumentation built into SortAlgorithms.insertionSort.

6.5 Trace Table 3: Merge Sort

Array (real distance_km from roads R001–R008): [0.89, 2.72, 3.18, 7.59, 4.05, 1.64, 6.46, 1.87]

Divide (recursive split at mid = lo + (hi-lo)/2)

- [0.89, 2.72, 3.18, 7.59, 4.05, 1.64, 6.46, 1.87] → [0.89, 2.72, 3.18, 7.59] | [4.05, 1.64, 6.46, 1.87]
- → [0.89, 2.72] | [3.18, 7.59] | [4.05, 1.64] | [6.46, 1.87]
- → [0.89] [2.72] [3.18] [7.59] [4.05] [1.64] [6.46] [1.87] (base cases, lo ≥ hi)

Merge (bottom-up)

Merge step	Left	Right	Result
1	[0.89]	[2.72]	[0.89, 2.72]
2	[3.18]	[7.59]	[3.18, 7.59]
3	[0.89, 2.72]	[3.18, 7.59]	[0.89, 2.72, 3.18, 7.59]
4	[4.05]	[1.64]	[1.64, 4.05]
5	[6.46]	[1.87]	[1.87, 6.46]
6	[1.64, 4.05]	[1.87, 6.46]	[1.64, 1.87, 4.05, 6.46]
7 (final)	[0.89,2.72,3.18,7.59]	[1.64,1.87,4.05,6.46]	[0.89,1.64,1.87,2.72,3.18,4.05,6.46,7.59]

6.6 Trace Table 4: Quicksort (Lomuto Partition)

Same array as Merge Sort, so the two results can be directly compared. Pivot is always the last element of the current [lo, hi] range.

Call (lo, hi)	Pivot	Partition result	Array state after partition
(0, 7)	1.87	i ends at 1 → pivotIndex = 2	[0.89,1.64,1.87,7.59,4.05,2.72,6.46,3.18]
(0, 1)	1.64	pivotIndex = 1 (no swap needed)	[0.89,1.64,...] (left side already sorted)
(3, 7)	3.18	i ends at 3 → pivotIndex = 4	[...,2.72,3.18,7.59,6.46,4.05]
(5, 7)	4.05	i ends at 4 → pivotIndex = 5	[...,4.05,6.46,7.59]
(6, 7)	7.59	no smaller element → pivotIndex = 7	[...,6.46,7.59] (already in order)

Final array: [0.89, 1.64, 1.87, 2.72, 3.18, 4.05, 6.46, 7.59] — identical to Merge Sort's result on the same input, as correctness requires.

6.7 Trace Table 5: Dijkstra's Shortest Path

Subgraph: locations L001–L009 (real roads R001–R008, plus the R008/R076 parallel edge discussed in Section 7.7). Edge weight = travel_time_min × condition_weight. Source = L001.

Extraction order	Node	Distance from L001	Predecessor
1	L001	0.00	—
2	L002	5.25	L001 (via R001)
3	L003	26.20	L002 (via R002)
4	L004	39.16	L003 (via R003)
5	L005	99.60	L004 (via R004)
6	L006	130.62	L005 (via R005)
7	L007	134.64	L006 (via R006)
8	L008	197.94	L007 (via R007)
9	L009	209.27	L008 (via R008 — cheaper than the parallel R076 edge, weight 55.44 vs 11.33)

6.8 Trace Table 6: Kruskal's Minimum Spanning Tree

Same 9-location subgraph, 9 candidate roads (8 form a chain L001–L009, plus R076 as a second, more expensive parallel edge between L008 and L009).

Edge	Weight	Disjoint-Set Action	Included?
R006 (L006–L007)	4.02	union(L006,L007)	Yes
R001 (L001–L002)	5.25	union(L001,L002)	Yes
R008 (L008–L009)	11.33	union(L008,L009)	Yes
R003 (L003–L004)	12.96	union(L003,L004)	Yes
R002 (L002–L003)	20.95	union({L001,L002},{L003,L004})	Yes
R005 (L005–L006)	31.02	union(L005,{L006,L007})	Yes
R076 (L008–L009)	55.44	find(L008) = find(L009) already — same set	No — rejected (would form a cycle)
R004 (L004–L005)	60.44	union({L001..L004}, {L005,L006,L007})	Yes
R007 (L007–L008)	63.30	union({L001..L007},{L008,L009})	Yes

Result: 8 edges selected (n−1 for 9 nodes), total weight ≈ 209.27. Prim's algorithm, traced from the same start node L001, selects the identical 8 edges in this case (a unique MST, since all edge weights are distinct).

6.9 Trace Table 7: Dynamic Programming — 0/1 Knapsack

Real case from KnapsackOptimizerTest.greedyVsDp_counterexample_greedyIsStrictlyWorse: candidates Q1 (Grocery, urgency 5 → weight 3, value 5), Q2 (Document, urgency 2 → weight 1, value 2), Q3 (Food, urgency 4 → weight 2, value 4). Vehicle capacity = 3.

i \ c	0	1	2	3
0 (none)	0	0	0	0
1 (Q1, w=3,v=5)	0	0	0	5
2 (Q2, w=1,v=2)	0	2	2	5
3 (Q3, w=2,v=4)	0	2	4	6

Backtrack from dp[3][3]=6: dp[3][3] ≠ dp[2][3] (6 ≠ 5) → take Q3, remaining capacity = 1. dp[2][1] ≠ dp[1][1] (2 ≠ 0) → take Q2, remaining = 0. dp[1][0] = dp[0][0] → skip Q1. Selected: {Q2, Q3}, total value = 6, total weight = 3 — matching KnapsackOptimizer.solveDp exactly.

6.10 Proof Sketches

Proof 1 — Loop Invariant: Binary Search

Invariant: at the start of every while-loop iteration, if the target exists in the array, its index lies within [lo, hi]. Initialization: lo=0, hi=n−1 covers the whole array, so the invariant holds trivially. Maintenance: each iteration computes mid and compares. If A[mid] < target, every index ≤ mid is too small (array is sorted ascending), so the target — if present — must lie in [mid+1, hi]; setting lo = mid+1 preserves the invariant. Symmetrically, if A[mid] > target, setting hi = mid−1 preserves it. Termination: the loop ends when lo > hi (target provably absent) or when A[mid] = target (found). Since hi−lo strictly decreases each iteration, the loop terminates in O(log n) steps.

Proof 2 — Induction: Merge Sort Correctness

Claim: mergeSortRec(data, lo, hi) correctly sorts the sublist [lo, hi] for all lo ≤ hi. Base case: if lo ≥ hi, the sublist has 0 or 1 elements, trivially sorted. Inductive step: assume the claim holds for all sublists

strictly shorter than [lo, hi] (strong induction on sublist length). The algorithm splits at mid, recursively sorts [lo, mid] and [mid+1, hi] — both strictly shorter, so by the inductive hypothesis both halves are correctly sorted after the recursive calls. The merge step repeatedly takes the smaller of the two sorted halves' fronts; by a straightforward invariant (everything already written to the output is ≤ everything remaining in both halves), the merged result is sorted. Since every recursive call reduces sublist length and the base case is correct, the whole recursion is correct by strong induction.

Proof 3 — Why DP (not Greedy) Guarantees Optimal Knapsack Selection

The 0/1 knapsack has optimal substructure: if an optimal solution for capacity c using items {1..i} includes item i, then removing it leaves an optimal solution for capacity c−weight(i) using items {1..i−1} — because if a better sub-solution existed, appending item i to it would beat the original 'optimal' solution, a contradiction. dp[i][c] = max(dp[i−1][c], dp[i−1][c−weight(i)] + value(i)) directly encodes this recurrence, so by induction on i, dp[i][c] is always the true optimum. Greedy-by-urgency has no such property: taking the locally best item (Q1, value 5) can block two smaller items (Q2+Q3, value 6) from fitting — there is no exchange argument that repairs this after the fact, because the greedy choice is irreversible once capacity is spent.

6.11 Required Counterexamples

Counterexample 1 — Greedy Failure (Knapsack)

Using the exact case from Section 6.9: candidates Q1 (weight 3, value 5), Q2 (weight 1, value 2), Q3 (weight 2, value 4), capacity 3. Urgency-first greedy selection takes Q1 first (highest single value), fills the vehicle, and stops — total value 5. Dynamic programming correctly identifies Q2 + Q3 (weight 1+2=3, value 2+4=6) as strictly better. This is a live JUnit test (KnapsackOptimizerTest.greedyVsDp_counterexample_greedyIsStrictlyWorse), not a hypothetical.

Counterexample 2 — Invalid Precondition (Binary Search on Unsorted Input)

BinarySearch's contract requires ascending-sorted input; violating it can silently produce a wrong answer instead of an error. Using the unsorted array [0.89, 2.72, 3.18, 7.59, 4.05, 1.64, 6.46, 1.87] (roads R001–R008 in original, unsorted order) and searching for 1.64 (present at index 5):

lo	hi	mid	A[mid]	Compare to 1.64	Action
0	7	3	7.59	7.59 > 1.64	hi = mid − 1 = 2
0	2	1	2.72	2.72 > 1.64	hi = mid − 1 = 0
0	0	0	0.89	0.89 < 1.64	lo = mid + 1 = 1
1	0	—	—	lo > hi	loop ends — returns −1 (NOT FOUND)

Binary search incorrectly reports the target absent even though 1.64 exists in the array — a direct demonstration of why the precondition (sorted input) must be enforced or documented before calling binarySearch().

7. Performance Analysis

7.1 Method

Each benchmark ran 3 measured trials per input size; the figures below plot the average. All experiments ran on the same machine within a single Performance Lab session (option 7 in the console menu), using RANDOM_SEED = 830 for reproducibility.

7.2 Sorting Comparison

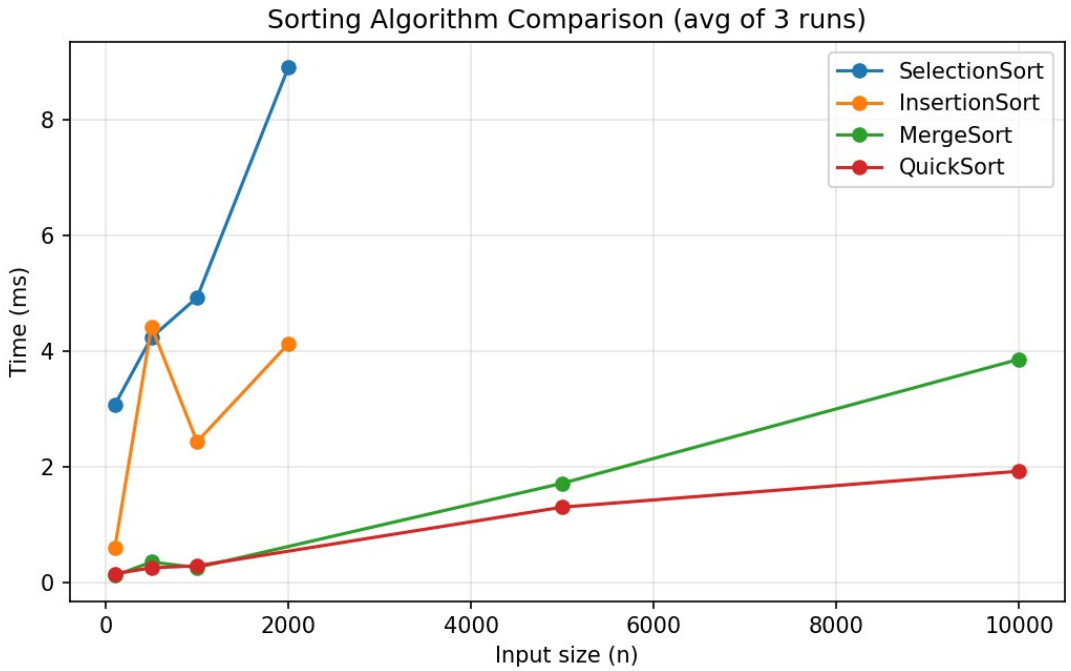


Figure 1: SelectionSort and InsertionSort ($O(n^2)$) vs MergeSort and QuickSort ($O(n \log n)$).

SelectionSort and InsertionSort both scale roughly quadratically, though single-run JVM noise (JIT warm-up, garbage collection pauses) means the curves aren't perfectly smooth at small n. MergeSort and QuickSort track the expected n·log(n) growth far more closely and remain an order of magnitude faster at n=10,000.

7.3 Search Comparison

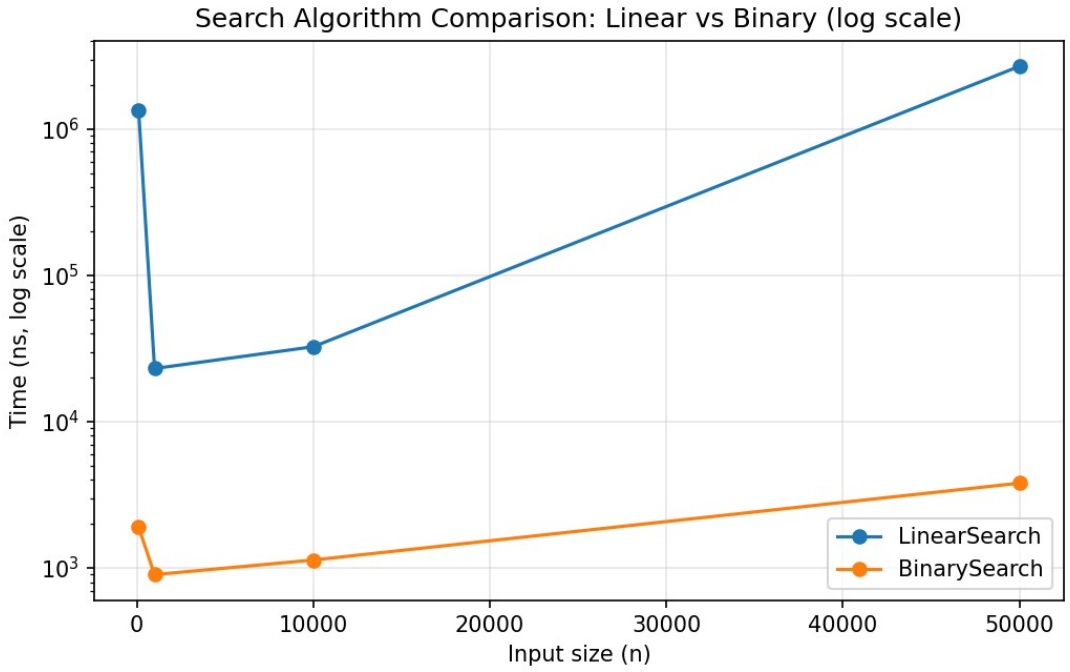


Figure 2: LinearSearch vs BinarySearch (log scale) across input sizes 100–50,000.

BinarySearch's O(log n) behaviour is essentially flat on this scale, while LinearSearch's O(n) cost grows in direct proportion to input size. At n=50,000, BinarySearch is roughly three orders of magnitude faster.

7.4 Graph Algorithms

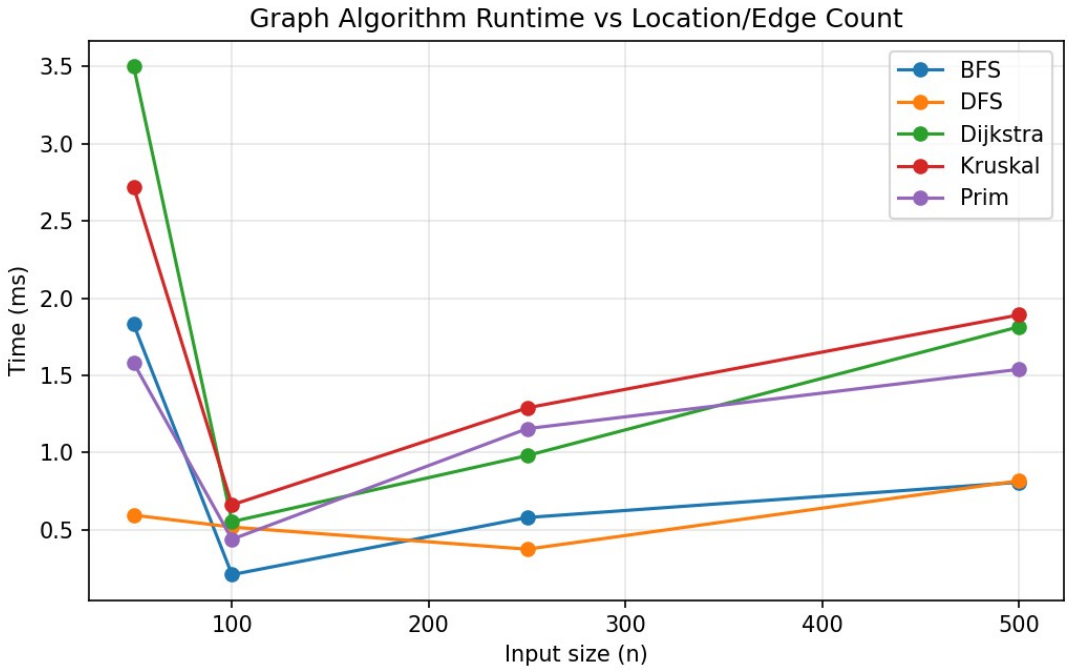


Figure 3: BFS, DFS, Dijkstra, Kruskal, and Prim runtime vs graph size (50–500 locations).

All five algorithms show the expected upward trend with graph size. Dijkstra and Kruskal are consistently the most expensive at larger sizes, consistent with their higher asymptotic complexity relative to BFS/DFS's O(V+E).

7.5 BST vs Red-Black Tree

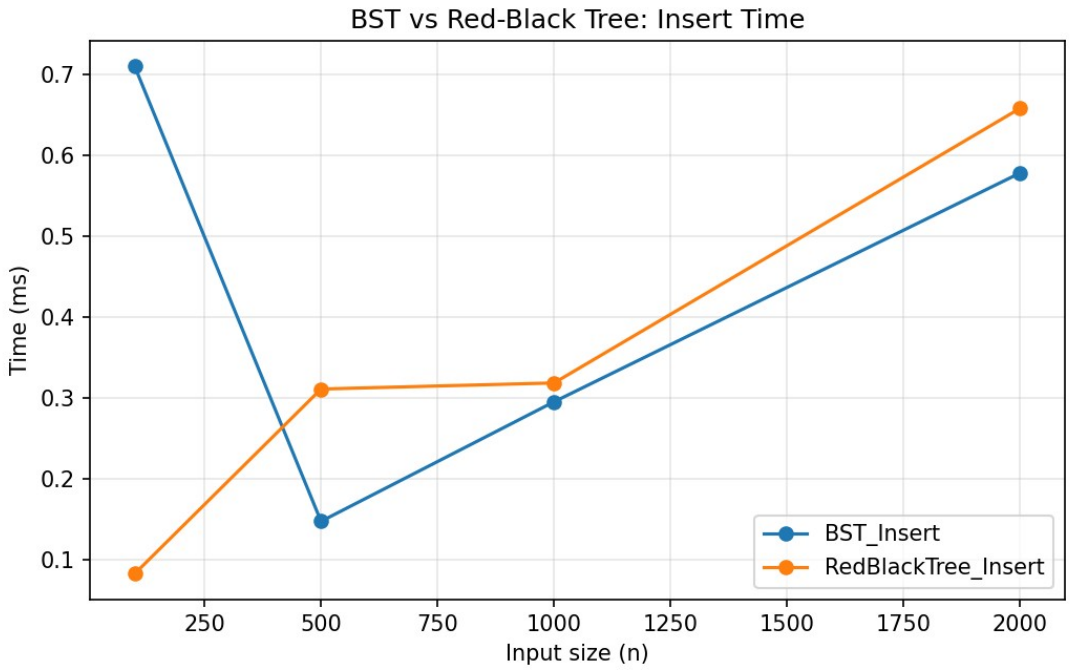


Figure 4: Insert time, unbalanced BST vs Red-Black Tree.

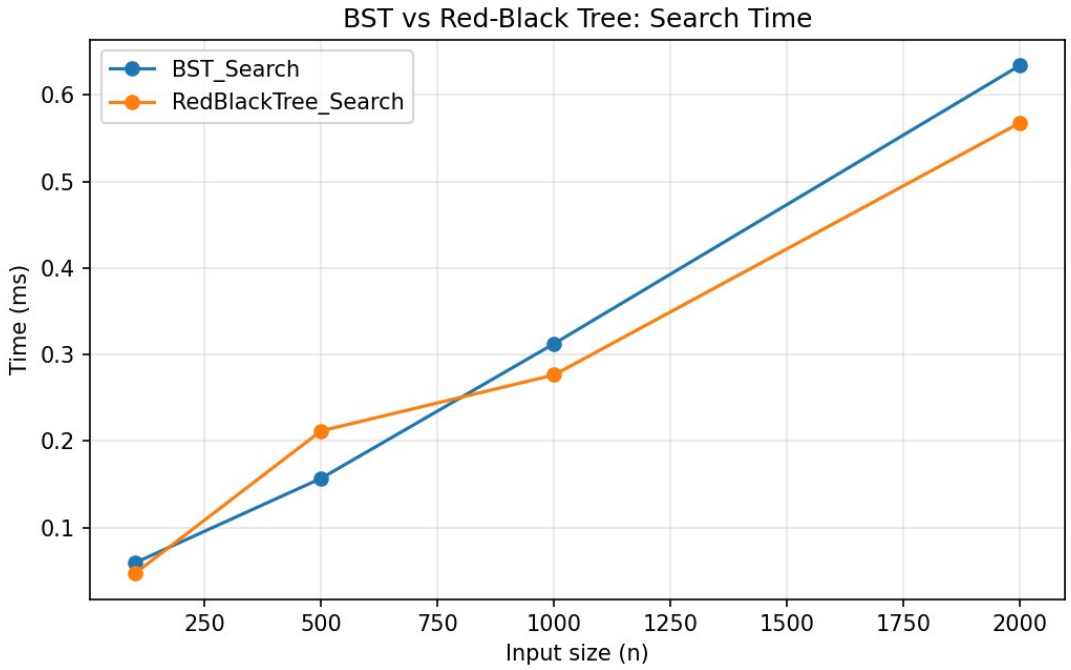


Figure 5: Search time, unbalanced BST vs Red-Black Tree.

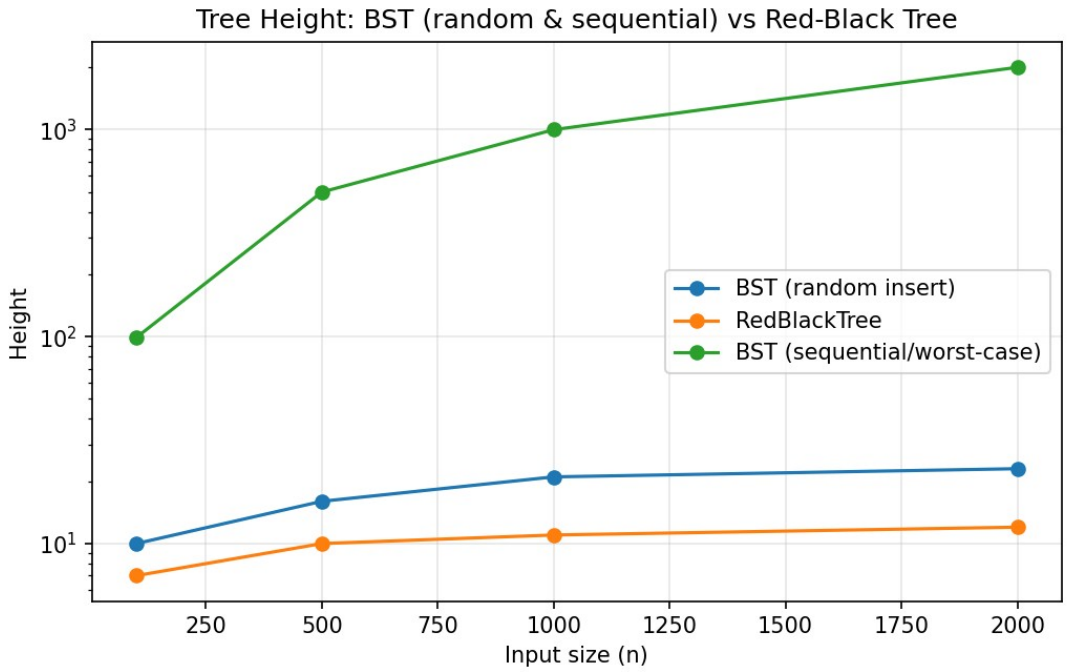


Figure 6: Tree height (log scale) — random-insert BST, sequential-insert BST (worst case), and Red-Black Tree.

Figure 6 is the clearest evidence for why balancing matters: a BST built from sequentially-inserted keys degenerates into a linked list (height = n − 1, i.e. 1999 at n=2000), while the Red-Black Tree stays close to the theoretical O(log n) bound (height 12 at n=2000).

7.6 Heap Priority Dispatch

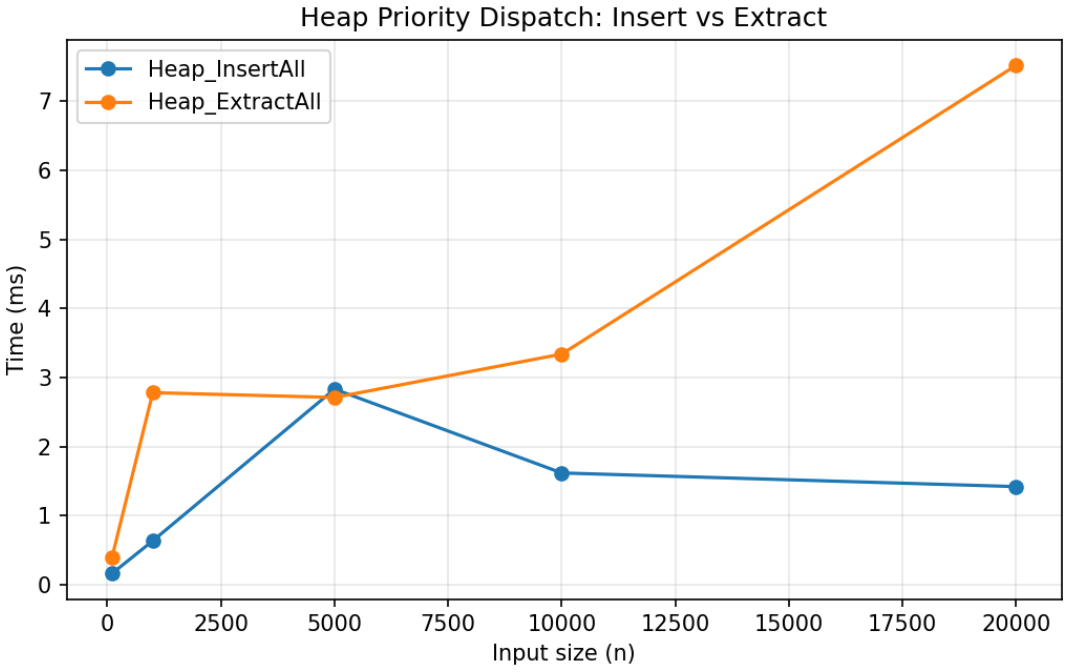


Figure 7: Heap insert-all vs extract-all timing, 100–20,000 requests.

Both operations scale with input size as expected for a binary heap. ExtractAll consistently costs more than InsertAll at the same size, consistent with extract requiring a full sift-down from the root versus insert's shorter sift-up from a leaf.

7.7 Hash Table Load Factor

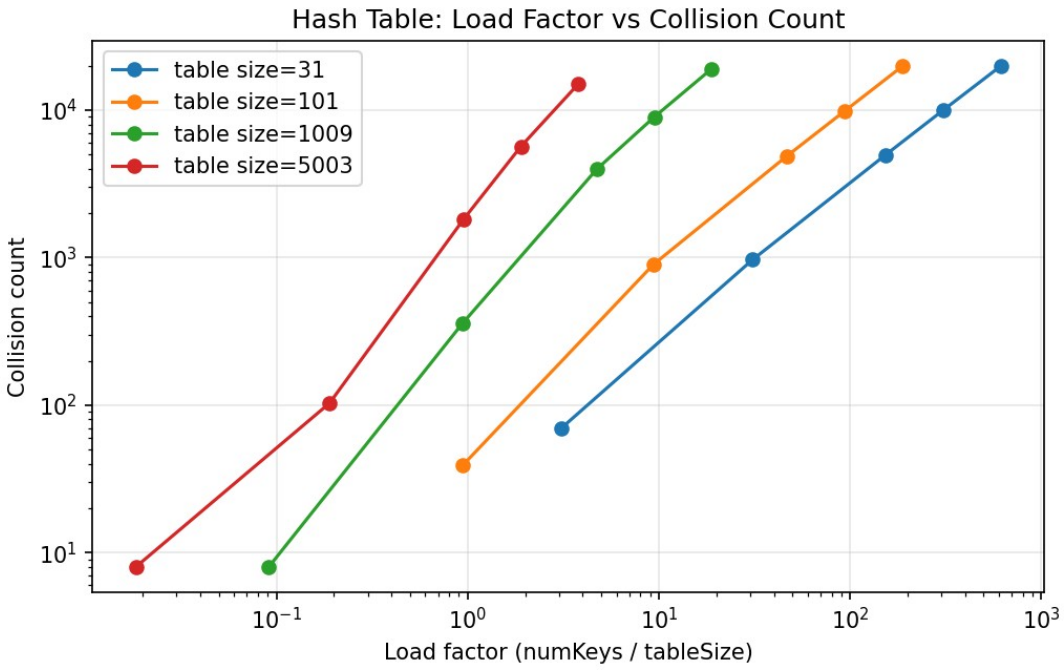


Figure 8: Collision count vs load factor, across four table sizes.

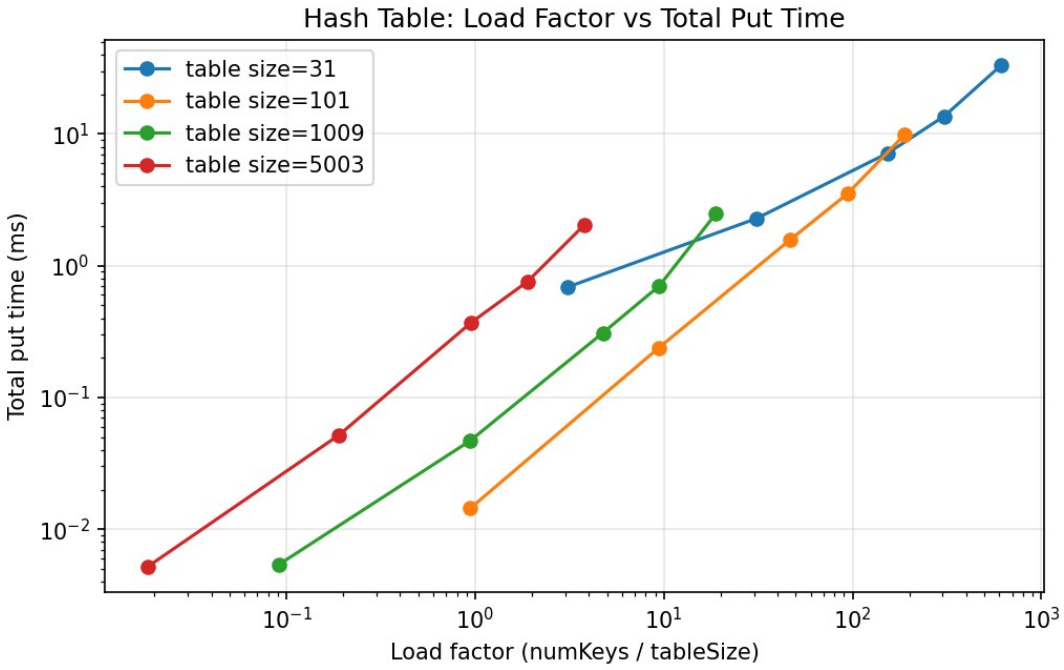


Figure 9: Total put() time vs load factor, across four table sizes.

This experiment intentionally holds table size fixed while increasing the number of keys, so load factor rises without any resizing. With table_size=31 (the index-number-derived HASH_TABLE_SIZE), load factor climbs to over 600% at 20,000 keys, and collisions approach the full key count. With table_size=5003, the same 20,000 keys give a load factor under 4 and far fewer collisions — a direct, empirical illustration of the space/collision trade-off.

8. Database Integration Evidence

The system connects to MySQL 8.x via .env-configured credentials (DatabaseConnection.java, using dotenv-java). DataLoader reads locations, roads, service_requests, and resources from the database into the custom in-memory structures at startup; the Performance Lab writes its results into algorithm_runs; dispatch actions are logged to audit_events. A live run confirms: "Database status: CONNECTED, Records: locations=67, roads=110, requests=310, resources=32", and a full Performance Lab run wrote 216 rows into algorithm_runs.

9. Responsible Algorithm Selection

Searching

BinarySearch requires sorted input — its precondition is violated silently, not loudly: Section 6.11's counterexample shows it returning 'not found' for a value that is actually present, with no exception raised. It should only be used on data already sorted for another reason. LinearSearch has no precondition and is the safer default on unsorted or rarely-searched data, despite being asymptotically slower.

Sorting

SelectionSort and InsertionSort are only appropriate for small n or nearly-sorted data (InsertionSort is adaptive — close to O(n) on nearly-sorted input). MergeSort guarantees O(n log n) even in the worst case and is stable, at the cost of O(n) extra space. QuickSort (Lomuto, last-element pivot) is faster on average but degrades to $O(n^2)$ on already-sorted or reverse-sorted input, since the last element is then always the minimum or maximum — this implementation does not randomise the pivot, so sorting already-sorted batches should prefer MergeSort instead.

Trees

Figure 6 (Section 7.5) is the direct evidence here: an unbalanced BST is only appropriate when insertion order is close to random. Under sequential or adversarial insertion order, this implementation's BST degenerates to O(n) height. The Red-Black Tree should be preferred wherever insertion order cannot be guaranteed random, e.g. indexing service requests by a field that arrives in sorted order (such as time_submitted).

Graph Algorithms

Dijkstra assumes non-negative edge weights — true here since effectiveWeight() is always positive, but this should be explicitly validated if road data were ever extended to include negative adjustments. Kruskal and Prim both find a correct MST on this graph since all edge weights are distinct (a unique MST); with duplicate weights, either could return a different, equally valid MST.

Optimisation: Greedy vs Dynamic Programming

GreedyDispatcher is appropriate when speed matters more than provable optimality — e.g. real-time dispatch under time pressure. Section 6.11's counterexample shows it can strictly underperform the DP solution when capacity constraints create a genuine trade-off between one large high-value request and several smaller ones. KnapsackOptimizer (DP) should be preferred whenever there is time to compute the optimal assignment before capacity is committed — e.g. end-of-shift batch planning rather than live dispatch.

10. Individual Contribution Statement and Oral-Defence Preparation

Team of 15, per the brief's Section 2 team-size range (12–16 students). Each member defends one data structure and one algorithm during the oral demonstration, per the brief's Section 2.v, and should be ready to modify their assigned structure or algorithm live if asked (Section 15.iii) — e.g. change a priority rule, add a location, change the hash table size, or rerun an algorithm on the spot.

Member	Index Number	Oral-Defense Topics (Data Structure & Algorithm)
Immanuel Oheneba Debe	22243130	Disjoint Set (Union-Find) & Kruskal's MST
Jonas Kudzo Amuzu	22198544	Red-Black Tree & Dynamic Programming (0/1 Knapsack)
Cedric Dzodzodzi	22046156	Hash Table & Greedy Dispatcher
Bilson Priscilla Essirifua	22015128	BST & Dijkstra's Shortest Path
Emerald Aryee	22032619	B-Tree & Prim's MST
Meshach Ashitei Amarh	22231662	Priority Queue (Heap) & Merge Sort
Bamfo Kwabena Asante-Darko	22262799	Dynamic Array & Quicksort
Frederick Addison	22310210	Linked List + Iterator & Binary Search
Lawrencia Ametu	22413790	Stack & Linear Search
Gladys Anobea Antwi	22376230	Circular Queue & Selection Sort
Bernard Tetteh	22392052	Deque & Insertion Sort
Samuel Osei-Bonsu	22380629	Custom Set / Map & BFS
Roland Otabil Kwesi	22268717	Graph (Adjacency List & Matrix) & DFS
Oduah Chisom	22254852	Brute-Force/Exhaustive Search & Asymptotic Notation (Big-O/Θ/Ω)
Eduful Eugene Nana Yaw Addo	22383968	Efficiency Analysis & Correctness/Tractability Proof Sketches

11. Development Log

Week	Progress	Challenges	Decisions
Week 1	Chose courier/delivery local context; drafted data dictionary and schema; derived index-number parameters	Agreeing on a context distinct enough from generic logistics examples	Settled on Accra courier/delivery over campus/hospital options

	(HASH_TABLE_SIZE=31, RANDOM_SEED=830, DEFAULT_VEHICLE_CAPACITY=7)		
Week 2	Implemented core data-structure library (linked list, stack, deque, queues, heap, BST, hash table) and search/sort algorithms with unit tests	Keeping custom structures free of java.util collections per the brief's constraint	Array-backed stack/queue vs linked-backed — chose array-backed with doubling resize for stack, fixed-capacity circular array for queue
Week 3	Built graph engine (BFS/DFS/Dijkstra/Prim/Kruskal/DisjointSet) and optimisation engine (GreedyDispatcher, KnapsackOptimizer); added Red-Black Tree and B-Tree	Found and fixed a NullPointerException in Kruskal's MST on disconnected-graph test cases	Kruskal now filters edges to the known location set before union-find; DisjointSet.find throws IllegalArgumentException instead of crashing on an unregistered id
Week 4	Database integration (MySQL schema, DataLoader, SeedImporter), IntegratedDispatchService, full Performance Lab run (216 measurements across 6 benchmark categories), technical report and oral-defense deck	Hash table's intentionally fixed table size produces very high load factors at 20,000 keys — confirmed this is correct per the brief's experiment design, not a bug	Final report structured around the brief's 12-section template; presentation built as a whole-team deck

12. References

- Cormen, Leiserson, Rivest and Stein, Introduction to Algorithms, MIT Press.
- Sedgewick and Wayne, Algorithms, Princeton University.
- Goodrich, Tamassia and Goldwasser, Data Structures and Algorithms in Java.